

IoT mini project final document

**IoT-Based Smart Classroom Noise Monitoring
System**



Contents

Table of figures.....	ii
1. Abstract.....	1
2. Introduction.....	1
2.1 Background of IoT and Project Context.....	1
2.2 Objectives and Goals of the Project	1
2.3 Scope and Limitations	2
3. Literature Review	2
4. Methodology.....	3
4.1 Description of Hardware and Software Used	3
4.2 System Architecture Diagram	3
4.3 Circuit Diagram	4
4.4 Data Collection, Processing, and Analysis.....	4
5. Implementation	5
5.1 Detailed Explanation of the IoT System Implementation.....	5
5.2 Challenges Faced and How They Were Addressed.....	6
5.3 Code Snippets.....	7
5.3.1 Wi-Fi and Firebase Connection.....	7
5.3.2 Noise Detection Using MAX9814 Sensor	8
5.3.3 Noise Classification and Alert Generation	8
5.3.4 Uploading Data to Firebase.....	9
6. Results	10
6.1 System Output.....	10
6.2 Analysis of Results.....	11
7. Discussion.....	11
8. Conclusion	11
9. Future Work	12
10. References.....	14

Table of figures

Figure 1 System Architecture Diagram	3
Figure 2 Circuit Diagram	4
Figure 3 Wi-Fi and Firebase Connection.....	7
Figure 4 Noise Detection Using MAX9814 Sensor.....	8
Figure 5 Noise Classification and Alert Generation	8
Figure 6 Uploading Data to Firebase	9
Figure 7 OLED Display Output	9
Figure 8 Display output of the system	10
Figure 9 Generate alert when high noise detection.....	10
Figure 10 Mobile interface and database.....	10

1. Abstract

The Internet of Things (IoT) is a modern technology that connects physical devices to the internet for collecting and sharing data. This project presents an IoT-Based Smart Classroom Noise Monitoring System developed to monitor classroom noise levels automatically. The system helps maintain a better learning environment by detecting excessive noise and providing alerts in real time.

The system was developed using an ESP32 microcontroller, MAX9814 sound sensor, OLED display, LEDs, buzzer, Flutter mobile application, and Firebase Realtime Database. The sound sensor detects classroom noise levels, and the ESP32 processes the data. According to the detected noise level, LEDs indicate quiet, medium, or loud conditions, while the buzzer alerts users when the noise exceeds the threshold level. The collected data can also be monitored remotely through the mobile application.

2. Introduction

2.1 Background of IoT and Project Context

Internet of things (IoT) is a technology where smart devices are able to exchange information through the internet. IoT applications are commonly found in the health sector, industrial operations, transportations, agriculture, and education institutions among others.

In the learning environment, there is always a need to create a conducive environment for learning. However, keeping track of the noise level in a classroom is usually challenging. Hence, this research paper presents the development of an IoT application aimed at monitoring classroom noise.

2.2 Objectives and Goals of the Project

The main objectives of this project are:

- To measure classroom noise levels using a sound sensor
- To monitor noise levels in real time
- To generate alerts when noise exceeds acceptable levels
- To display noise conditions using LEDs and OLED display
- To store monitoring data in Firebase Realtime Database
- To allow remote monitoring using a Flutter mobile application

2.3 Scope and Limitations

Scope

- Classroom noise monitoring
- Real-time alert generation
- Cloud-based data storage
- Mobile application monitoring

Limitations

- Requires internet connection for Firebase communication
- Accuracy may vary depending on environmental conditions
- System was tested only in small classroom environments

3. Literature Review

Smart technology is employed by many schools for better class management and enhanced learning experience. Real-time data collection and remote monitoring features have made IoT monitoring systems common.

A number of similar projects have been implemented previously using sound sensors along with microcontrollers such as Arduino and ESP32 to collect data for noise monitoring. Cloud platforms like Firebase are often employed to store data collected through sensors, whereas apps on smartphones are deployed for remote monitoring.

As compared to conventional approaches to monitoring, systems based on IoT are efficient due to their reduced dependence on human labor. The current project utilizes a combination of ESP32, Firebase, and Flutter technologies.

4. Methodology

4.1 Description of Hardware and Software Used

An IoT-Based Smart Classroom Noise Monitoring System has been implemented through the use of both hardware and software components. The main hardware component used in the implementation of the project includes the ESP32 microcontroller which acted as the processing unit of the whole system. The MAX9814 sound sensor was used to detect noise levels in the classroom. The OLED SSD1306 screen was used to display the current noise levels and system status. LEDs in green, yellow, and red colors were used to detect different noise levels while the buzzer was used to produce an alarm if there is too much noise.

The software side of the project was implemented by programming the ESP32 microcontroller using the Arduino IDE. The remote monitoring of the system has been achieved by creating a Flutter mobile application for the same purpose. Firebase Realtime Database was used as the cloud platform.

4.2 System Architecture Diagram

There are different layers in the system architecture used for this project. In the input layer, there is the MAX9814 sound sensor that senses data on sounds from the classroom environment. In the processing layer, we have the ESP32 microcontroller that processes the data sensed by the sensor and compares it against threshold values. In the output layer, there is the OLED display, LED light, and buzzer that give alert notifications depending on the amount of noise. In the cloud layer, Firebase Realtime Database stores the data collected. Lastly, there is the user layer where the data can be remotely accessed using the mobile Flutter application.

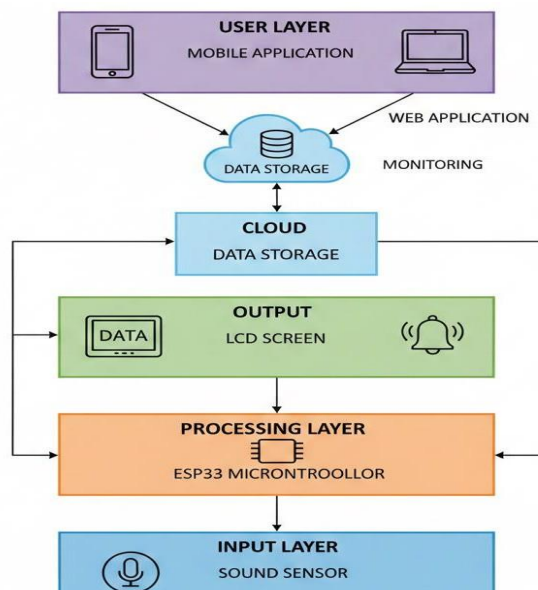


Figure 1 System Architecture Diagram

4.3 Circuit Diagram

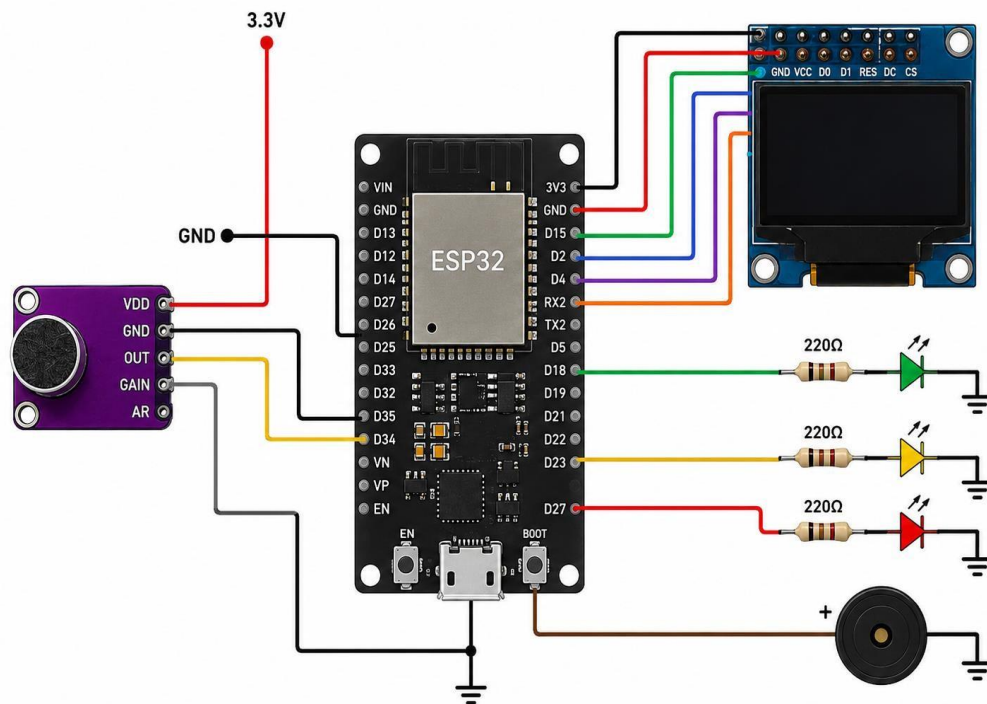


Figure 2 Circuit Diagram

4.4 Data Collection, Processing, and Analysis

The process entails continuous acquisition of sound data through MAX9814 sound sensors. The sound sensors transmit sound data in the form of analog sound signals to the ESP32 microcontroller via GPIO34 port. In the processing phase, the ESP32 compares the acquired sound signals against pre-defined threshold levels.

If the sound signal does not exceed the quiet threshold, then the green LED indicator lights up as a representation that the environment is quiet. In the case where the sound value lies within the average noise level range, then the yellow LED will light up.

When the sound exceeds the loud threshold, then the red LED and buzzer are turned on as an alarm mechanism to notify users of noise pollution. In addition, data acquired after the processing phase, together with noise level and status indicators, is presented on the OLED display, as well as uploaded to Firebase Realtime Database.

5. Implementation

5.1 Detailed Explanation of the IoT System Implementation

The implementation of the IoT-Based Smart Classroom Noise Monitoring System comprised both hardware and software technologies. The system is capable of detecting classroom noise level and giving real-time noise monitoring.

In the beginning, hardware components were connected based on the design of the circuit. The MAX9814 sound sensor was connected to the ESP32 microcontroller on GPIO34 pins in order to receive the sound data. To display the noise level and monitor the status of the system, an OLED SSD1306 display was attached by SPI communication pins. The three LEDs were utilized for noise condition indication, and the buzzer was included to provide alerts in case of loud noise.

ESP32 microcontroller was programmed with the help of the Arduino IDE. In particular, the system continually read sound value via the microphone sensor and then compared it with the predetermined threshold values. Based on the result of comparison, the corresponding LED and the buzzer were activated.

- LED green – a silent environment
- LED yellow – a normal noise level
- LED red and buzzer – loud noise

Furthermore, the system also provides real-time data like the current noise level and system status on the OLED screen.

Firebase Realtime Database was utilized for the integration of cloud services for storing the monitoring data. The ESP32 device can connect to Wi-Fi and send the collected data of sound level and system status to Firebase. A mobile app with Flutter was also built for getting and displaying this data on the cloud.

The IoT-based noise monitoring system was successfully implemented in the classroom environment.

5.2 Challenges Faced and How They Were Addressed

Various difficulties had to be overcome in the process of implementing the project.

- Firstly, it concerned the selection of the right connections between the components, especially the GPIO pin connections of the ESP32 board. At first, there were several errors as some devices refused to work properly. The solution was to double-check the ESP32 diagram and test connections separately.
- Next, connecting the ESP32 to Firebase Realtime Database proved to be a challenge as there were connection errors and wrong Firebase configuration options that did not let me upload any data into the Firebase server. All those problems were eliminated after checking the Firebase API Key, database URL, and Wi-Fi configurations.
- OLED display connectivity was also difficult because the SPI communication pins had to be configured correctly. This task was accomplished with the help of library documentation and separate testing of display functions.
- Unstable sensor readings had to be solved by setting up maximum sound values for a five-second period.

All of these problems were eventually solved and the system became more stable and reliable.

5.3 Code Snippets

5.3.1 Wi-Fi and Firebase Connection

The following code initializes the Wi-Fi connection and connects the ESP32 to the Firebase Realtime Database.

```
70 // WiFi
71 WiFi.begin(WIFI_SSID, WIFI_PASSWORD);
72
73 Serial.print("Connecting WiFi");
74
75 while (WiFi.status() != WL_CONNECTED) {
76     Serial.print(".");
77     delay(300);
78 }
79
80 Serial.println("\nWiFi Connected!");
81
82 // Firebase config
83 config.api_key = API_KEY;
84 config.database_url = DATABASE_URL;
85
86 if (Firebase.signUp(&config, &auth, "", "")) {
87     Serial.println("Firebase SignUp OK");
88 } else {
89     Serial.printf("SignUp Error: %s\n",
90                 config.signer.signupError.message.c_str());
91 }
92
93 Firebase.begin(&config, &auth);
94 Firebase.reconnectWiFi(true);
```

Figure 3 Wi-Fi and Firebase Connection

This code enables the ESP32 to communicate with the cloud database through a wireless network connection.

5.3.2 Noise Detection Using MAX9814 Sensor

The ESP32 continuously reads sound values from the MAX9814 microphone sensor and stores the highest value detected during a five-second interval.

```
113  
114     int soundValue = analogRead(MIC_PIN);  
115     if (soundValue > maxSound) {  
116         maxSound = soundValue;  
117     }
```

Figure 4 Noise Detection Using MAX9814 Sensor

The maximum sound value is used to represent the classroom noise level during each monitoring cycle.

5.3.3 Noise Classification and Alert Generation

The system compares the detected sound level with predefined threshold values and activates the appropriate indicators.

```
129     if (maxSound < QUIET_THRESHOLD) {  
130         statusText = "Quiet";  
131         digitalWrite(GREEN_LED, HIGH);  
132     }  
133     else if (maxSound < AVERAGE_THRESHOLD) {  
134         statusText = "Average";  
135         digitalWrite(YELLOW_LED, HIGH);  
136     }  
137     else {  
138         statusText = "Loud";  
139         digitalWrite(RED_LED, HIGH);  
140         digitalWrite(BUZZER_PIN, HIGH);  
141     }
```

Figure 5 Noise Classification and Alert Generation

Green, yellow, and red LEDs indicate different noise levels, while the buzzer is activated when excessive noise is detected.

5.3.4 Uploading Data to Firebase

The processed noise level and status are uploaded to Firebase Realtime Database for remote monitoring through the Flutter mobile application.

```
166     if (!Firebase.RTDB.setInt(&fbdo, "NoiseMonitor/SoundLevel", maxSound)) {  
167         Serial.println(fbdo.errorReason());  
168     }  
169  
170     if (!Firebase.RTDB.setString(&fbdo, "NoiseMonitor/Status", statusText)) {  
171         Serial.println(fbdo.errorReason());  
172     }
```

Figure 6 Uploading Data to Firebase

This allows real-time synchronization of classroom noise data between the ESP32 device and the mobile application.

5.3.5 OLED Display Output

The OLED display presents the current noise level and classroom status directly on the monitoring device.

```
149     display.clearDisplay();  
150     display.setTextSize(1);  
151     display.setTextColor(WHITE);  
152     display.setCursor(10, 5);  
153     display.println("Noise Monitor");  
154  
155     display.setCursor(10, 25);  
156     display.print("Level: ");  
157     display.println(maxSound);  
158  
159     display.setCursor(10, 45);  
160     display.print("Status: ");  
161     display.println(statusText);  
162  
163     display.display();
```

Figure 7 OLED Display Output

The display provides immediate visual feedback to users without requiring access to the mobile application.

6. Results

6.1 System Output

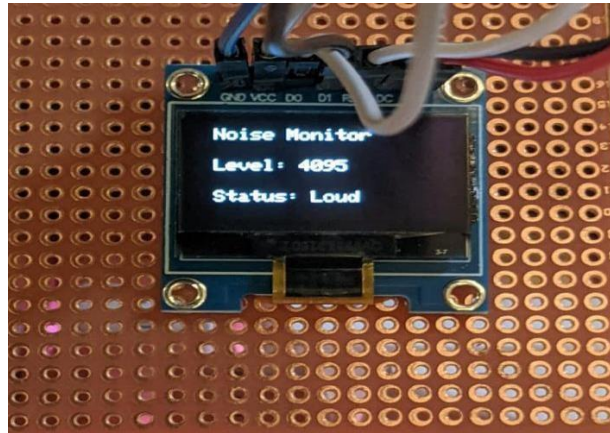


Figure 8 Display output of the system

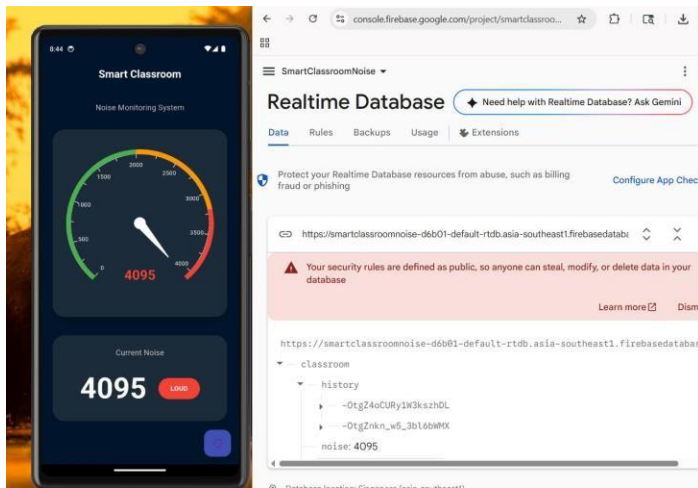


Figure 10 Mobile interface and database

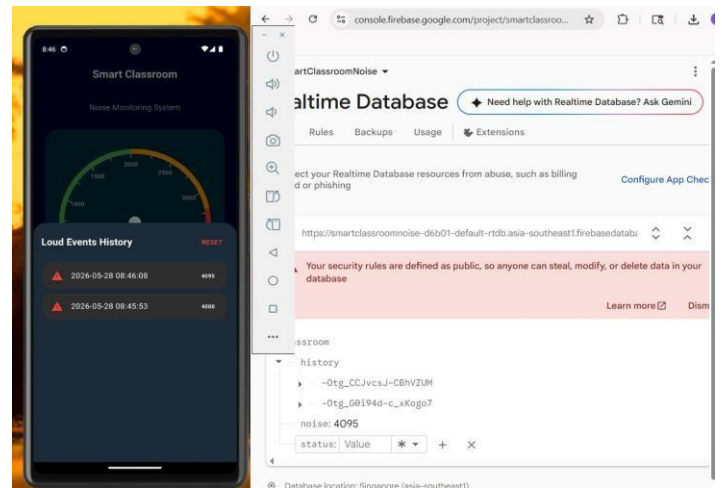


Figure 9 Generate alert when high noise detection

6.2 Analysis of Results.

The system operated well for all the noise levels and was able to provide alerts accordingly. The integration of Firebase was successful, which enabled us to monitor remotely using the Flutter app.

The primary goal of the project, which was to develop an IoT-enabled noise monitoring system for classrooms, was met successfully.

7. Discussion

The designed system enhanced the effectiveness of classroom noise monitoring through automatic detection of high noise levels in the classroom. The IoT technology-based method is better and faster than manual techniques.

The use of Firebase and Flutter increased the user-friendliness of the system. In addition to that, the project showed practical applications of IoT technologies in educational settings.

8. Conclusion

The IoT-based Smart Classroom Noise Monitoring System Project has been completed with great success, using devices like ESP32 microcontroller, MAX9814 sound sensor, Firebase Realtime Database, and Flutter mobile application.

The system was capable of:

- Monitoring classroom noises
- Creating alerts
- Showing live information
- Storing information online
- Remote monitoring

Thus, it can be concluded that the project has been a success, and its objectives have been met.

9. Future Work

Although the IoT-Based Smart Classroom Noise Monitoring System was successfully implemented, several improvements can be introduced in future developments to enhance the efficiency, scalability, and intelligence of the system.

- Mobile Notification System

In the current implementation, users can monitor classroom noise levels through the Flutter mobile application. However, future versions can integrate push notification services such as Firebase Cloud Messaging (FCM). This will allow teachers or administrators to receive instant alerts on their smartphones whenever the classroom noise exceeds the predefined threshold level.

- Multi-Classroom Monitoring

Currently, the system is designed for monitoring a single classroom environment. Future implementations can expand the system to support multiple classrooms simultaneously. A centralized cloud dashboard can be developed to monitor noise levels across different classrooms within a school or university.

- Web-Based Dashboard

A web-based administration dashboard can be developed for teachers and administrators. The dashboard can provide:

Real-time classroom monitoring

Historical noise analysis

Graphical reports and statistics

User management features

Device management options

This enhancement would improve usability and accessibility of the system.

- Improved Sensor Accuracy

The accuracy of the current system depends on environmental conditions and microphone sensitivity. Future improvements may include the use of advanced digital sound sensors with noise filtering and calibration mechanisms to obtain more stable and precise readings.

- Data Analytics and Reporting

Future systems can generate automated daily, weekly, and monthly reports regarding classroom noise levels. These reports can help school management identify classroom behavior patterns and improve learning environments.

- Battery Backup and Power Optimization

Currently, the system depends on a continuous power supply. Future versions can integrate rechargeable battery systems and power optimization techniques to ensure uninterrupted operation during power failures.

10. References

- [1] Espressif Systems, “ESP32 Series Datasheet,” Espressif Systems, 2026. [Online]. Available: <https://www.espressif.com/en/products/socs/esp32>. [Accessed: 28-May-2026].
- [2] Arduino, “Arduino IDE Documentation,” Arduino Official Documentation, 2026. [Online]. Available: <https://docs.arduino.cc/software/ide-v2>. [Accessed: 28-May-2026].
- [3] Google, “Flutter Documentation,” Flutter Official Documentation, 2026. [Online]. Available: <https://docs.flutter.dev/>. [Accessed: 28-May-2026].
- [4] Google Firebase, “Firebase Realtime Database Documentation,” Firebase Documentation, 2026. [Online]. Available: <https://firebase.google.com/docs/database>. [Accessed: 28-May-2026].
- [5] Adafruit Industries, “Adafruit SSD1306 OLED Display Library,” GitHub Repository, 2026. [Online]. Available: https://github.com/adafruit/Adafruit_SSD1306. [Accessed: 28-May-2026].
- [6] Adafruit Industries, “Adafruit GFX Graphics Library,” GitHub Repository, 2026. [Online]. Available: <https://github.com/adafruit/Adafruit-GFX-Library>. [Accessed: 28-May-2026].
- [7] Analog Devices, “MAX9814 Microphone Amplifier Datasheet,” Analog Devices Documentation, 2026. [Online]. Available: <https://www.analog.com/en/products/max9814.html>. [Accessed: 28-May-2026].
- [8] Cisco Networking Academy, “Introduction to IoT,” Cisco Networking Academy, 2026. [Online]. Available: <https://www.netacad.com/courses/iot/introduction-iot>. [Accessed: 28-May-2026].
- [9] Random Nerd Tutorials, “ESP32 Firebase Realtime Database Tutorial,” Random Nerd Tutorials, 2026. [Online]. Available: <https://randomnerdtutorials.com/esp32-firebase-realtime-database/>. [Accessed: 28-May-2026].
- [10] Syncfusion, “Flutter Radial Gauge Documentation,” Syncfusion Documentation, 2026. [Online]. Available: <https://help.syncfusion.com/flutter/radial-gauge/overview>. [Accessed: 28-May-2026].